LLM EVALUATION

DATASETS

METRICS

EVALUATION

ANALYSIS

EXPERIMENTS

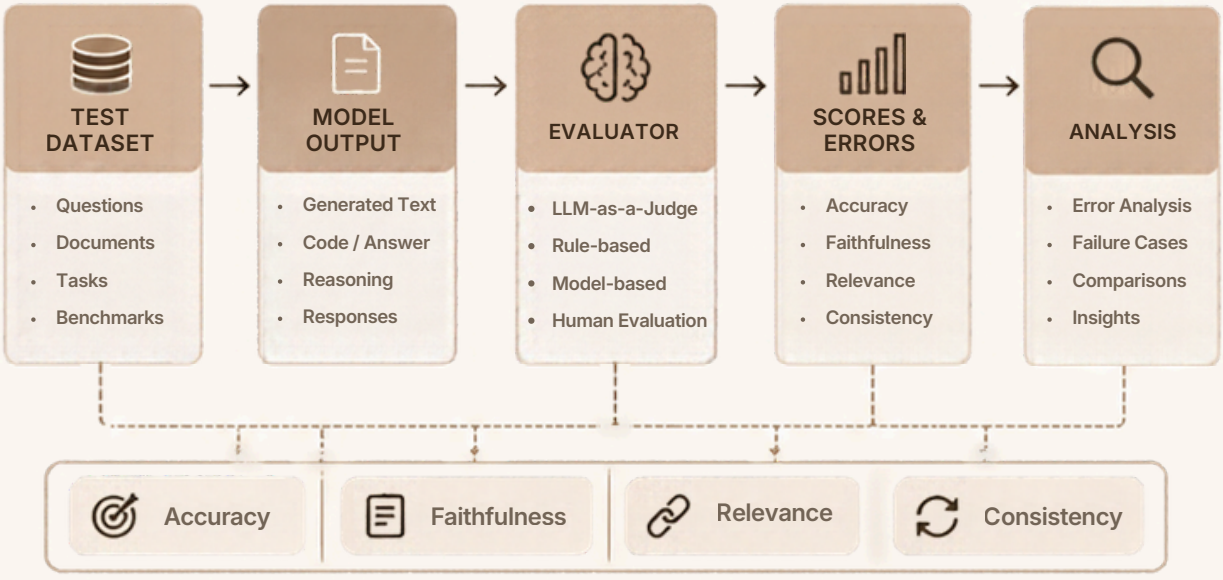
MEASURE
UNDERSTAND
IMPROVE

LLM EVALUATION · CHAPTER 03

LLM EVALUATION WORKFLOW

From Task Definition to Production Monitoring

A practical guide to success criteria, golden datasets, evaluation methods, iteration, and production monitoring.



WHAT THIS SERIES COVERS

- Evaluation Design
- Metrics & Rubrics
- LLM-as-a-Judge
- Regression & Error Analysis

PREMIUM REVISION NOTES

CONCEPTS · IMPLEMENTATION · DIAGRAMS · INTERVIEW PREP

LLM Evaluation Workflow

RECAP — CHAPTER 2

Earlier we covered *why* LLM evaluation matters and *what* LLM evals are, including their two types: **model evals** (judging a foundation model) and **application evals** (judging a system built on top of a model). This chapter moves to *how* evaluation is done, taught entirely from the **application-eval** perspective rather than the model-eval one.

1. The Running Example: Email Classification

The Scenario

The lecture uses one deliberately simple LLM application throughout. Imagine working as an engineer at a large food-delivery company (the lecture uses Zomato). The company receives a very large number of customer emails every day, and replying to or even sorting them by hand does not scale. The goal is to **automate** the sorting step.

The System

The system reads each email and classifies it from its content into one of three categories: a **billing** question, a **technical** problem, or a **general** query. The class decides where the email is routed: billing queries go to the billing team, technical queries to the technical team, and general queries to the customer support team. No person is needed to read emails and tag teams manually.

Implementation is minimal: a single LLM is given a system prompt telling it that it is a customer agent who reads an email and decides where it should be routed (Figure 3.1). The lecturer notes this can be built in 5–10 minutes.

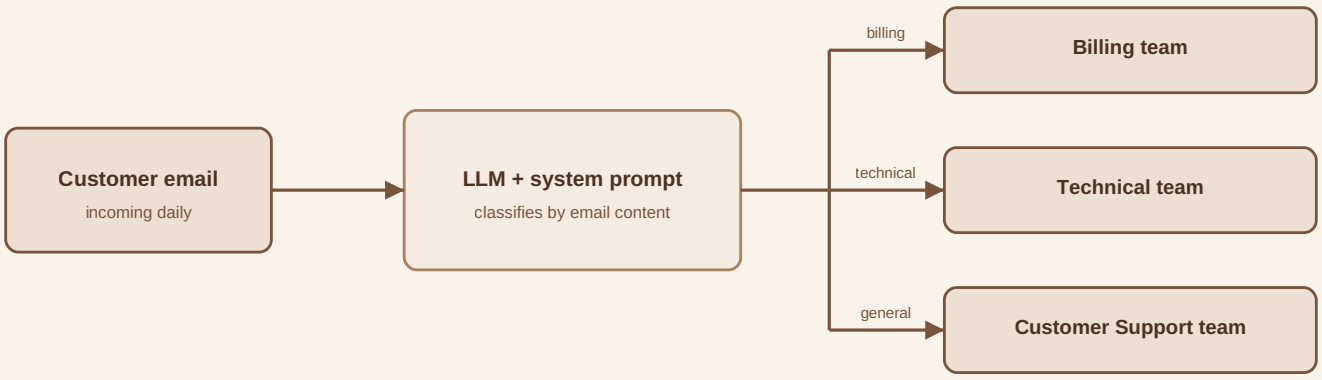


Figure 3.1 — The example application: one LLM classifies each customer email and routes it to the right team.

The Real Question

Because the system is so easy to build, the tempting next step is to deploy it directly. The lecture's answer is a firm **no**: the system must be **evaluated** first. The evaluation workflow below is introduced on this toy example, but it is the same workflow used throughout the course.

> **IMPORTANT**
>
> The workflow is not specific to classifiers. The lecture states that the same flow applies to RAG applications and to complex agents, so it is worth learning once, thoroughly, on the simple case.

> **↻ Quick Revision — The Example Application**
>
> - The app reads a customer email, classifies it as billing, technical, or general, and routes it to the matching team.
> - It is a single LLM with a system prompt, quick to build but not safe to deploy untested.
> - The lecture teaches the evaluation workflow from the application-eval perspective, not the model-eval one.
> - The same workflow carries over to RAG applications and agents.

2. The Nine-Step Evaluation Workflow

The workflow has nine steps. Steps 1–4 prepare the evaluation, steps 5–8 form a repeating loop, and step 9 continues after deployment. Figure 3.2 shows the flow; the table maps each step to the email example.

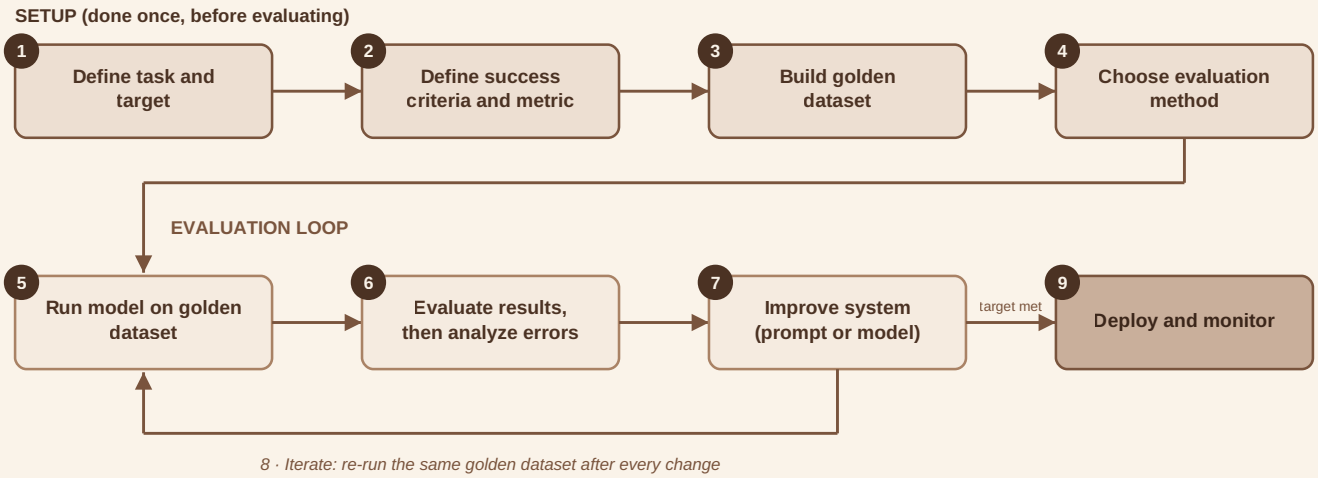


Figure 3.2 — Setup (steps 1–4), the evaluation loop (steps 5–8), and the hand-off to deployment and monitoring (step 9).

#	Step	In the email example
1	Define the task and target	Evaluate the whole email-routing system; the task is classification.
2	Define success criteria and metric	Correct classification, measured by accuracy.
3	Build a golden dataset	Emails paired with manually assigned labels.
4	Choose an evaluation method	Automated: a simple Python script.
5	Run the model on the dataset	Send every email through the system and collect its labels.
6	Evaluate and analyze results	Compute accuracy, then study where the errors occur.
7	Improve the system	Tweak the system prompt or change the LLM.
8	Iterate	Re-run the same dataset after each change.
9	Deploy and monitor	Watch production; add failures to the golden dataset.

INTERVIEW TIP

Being able to recite this loop end to end, with a concrete example such as the email router, is a strong answer to "How would you evaluate an LLM application?". Emphasize that it is a *loop*, not a one-time test.

↻ Quick Revision — The Workflow

- Nine steps: define task, define success criteria, build dataset, choose method, run, evaluate and analyze, improve, iterate, deploy and monitor.
- Steps 1–4 are setup; steps 5–8 repeat until the system is good enough to deploy.
- Evaluation does not end at deployment; monitoring feeds back into the dataset.

3. Steps 1–2: Define the Task, Target, and Success Criteria

Step 1 — Define the Task and Target

Decide *what* is being evaluated. Here the **target** is the whole system (the workflow that reads an email and routes it), and the **task** is a simple classification task: does the system classify emails correctly or not?

Step 2 — Define Success Criteria and Metrics

Next, decide how you will know the system works. For this use case the **success criterion** is correct classification, and the **metric** used to quantify it is **accuracy**. If the system receives 100 queries and routes 90 of them to the right place, it is 90% accurate.

Term	Meaning	Email example
Task	The kind of job being evaluated	Classification
Target	The thing under evaluation	The full email-routing system
Success criterion	What “working” means for this use case	Emails are classified correctly
Metric	The number that measures the criterion	Accuracy (90 of 100 correct = 90%)

> **COMMON MISTAKE**
>
> Skipping these two steps and jumping straight to running tests. Without a defined target and metric, a result such as “it seems fine” cannot be compared across iterations.

> ↻ **Quick Revision — Task and Success Criteria**
>
> • The target is what you evaluate (here, the whole system); the task is the kind of job (classification).
>
> • The success criterion states what “working” means; the metric turns it into a number.
>
> • For classification the lecture's metric is accuracy: correct predictions out of total, such as 90 of 100 = 90%.

4. Step 3: Build a Golden Evaluation Dataset

Definition

A **golden dataset** is a set of inputs paired with the correct answers, labelled manually by people. In the example, one column holds the email content and the other holds the true category assigned by hand.

Email content (input)	Manual label (expected output)
“My card was charged twice.”	Billing
“The app crashes on login.”	Technical
“What are your hours?”	General

This sample has three rows only to illustrate the idea. In practice the lecture suggests a dataset of roughly **50 to 500 rows**.

Where the Data Comes From

The best source is **real data**. For the food-delivery company, pull up past customer emails or chats and have someone label them manually. This labelled collection is what LLM-eval practice calls the golden dataset.

BEST PRACTICE

Prefer real historical data over invented examples, and have the labels assigned by a person, because the labels define what "correct" means for every later run.

↻ Quick Revision — Golden Dataset

- A golden dataset pairs inputs (email content) with manually assigned correct labels.
- Typical size in the lecture: about 50 to 500 rows.
- Best source: real past data (emails or chats), labelled by a person.
- The same dataset is reused across runs, which is what makes evals repeatable.

5. Step 4: Choose an Evaluation Method

What the Choice Means

This step decides **who or what performs the evaluation** once the system has produced answers: running the system over the dataset produces one prediction per row, and those predictions must be compared with the labels to compute the metric. There are three options.

Method	When it fits	Trade-off (as given in the lecture)
Automated (code)	Outputs can be compared mechanically, such as a short class label.	Cheap and fast; cannot judge whether two paragraphs mean the same thing.
Human	Outputs need judgement of meaning.	Costly, since a person must be paid, and scales poorly when much testing is needed.
LLM (another LLM as evaluator)	Long text outputs where code is not enough.	Sits in between: more scalable than people, more flexible than code.

Why Automated Wins Here

In the email example both the expected output and the system output are a single label, so a short Python script that counts matches and reports the accuracy score is enough. Paying a person would be wasteful, and an LLM is unnecessary.

When It Would Be Different

Imagine the application were a chatbot instead. The expected answer would be a long paragraph, and so would the LLM's answer. Code cannot reliably decide whether two paragraphs carry the same semantic meaning. A human could, but would be expensive when many tests are needed; the middle option is to use an LLM as the evaluator.

IMPORTANT

The evaluation method depends on the *shape of the output*. Label outputs allow automated checking; free-text outputs push you toward human or LLM evaluation.

↻ Quick Revision — Evaluation Method

- Three options: automated (code), human, or another LLM.
- Short label outputs, as in classification, can be scored by simple code.
- Long free-text outputs are hard to compare in code; humans are costly, so LLM evaluators are the middle ground.
- The email app uses the automated method: a Python script that computes accuracy.

6. Steps 5–6: Run the Model, Evaluate, and Analyze

Step 5 — Run the Model

"Running the model" means sending the golden dataset through the system you built. The system generates one answer per row, for example Billing for the first email, General for the second, and General for the third.

Step 6 — Evaluate and Analyze the Results

Using the chosen method, compare the system's answers with the manual labels and compute the metric. Suppose 100 rows are sent and 80 are classified correctly: accuracy is **80%** and 20 rows are wrong.

Evaluation alone is not enough. The next part is to **analyze** the result: think about *where* mistakes happen and what could be improved. This analysis tells you what to change in the next step.

BEST PRACTICE

Treat the accuracy number as the start of analysis, not the end. Look at which emails were misclassified before deciding what to change.

↻ Quick Revision — Run, Evaluate, Analyze

- Run: send the whole golden dataset through the system to get one prediction per row.
- Evaluate: compare predictions with labels, for example 80 correct out of 100 is 80% accuracy.
- Analyze: study the wrong cases to find where the system is failing.

7. Steps 7–8: Improve the System and Iterate

Step 7 — Improve the System

Once analysis shows where the system fails, change the **system**, not the dataset. The lecture gives two examples of levers:

- **The system prompt.** It may be written in a way that makes the LLM confuse billing and technical emails.
- **The LLM itself.** A smaller or open-source model with fewer parameters may simply not be strong enough for the task, so switching to a heavier model can help.

Step 8 — Iterate

After every change, **trigger the evaluation again** on the same golden dataset. This is what the lecture means when it says LLM evals are *repeatable*. The worked example runs as follows:

Round	Change made to the system	Accuracy
Baseline	Initial prompt and model	80%
After tweak	Prompt modified	90%
After change	Heavier LLM used	95%

The loop of build dataset, run model, evaluate, analyze, improve, and re-run continues until the team reaches a point where the system is judged **worth deploying**. In the lecture's story, the manager is happy at 95% and the iteration stops.

> **COMMON MISTAKE**
>
> Changing the evaluation data to make scores look better. The point of a fixed golden dataset is that every score is comparable with the last one; only the system changes between runs.

↻ Quick Revision — Improve and Iterate

- Improve the system, not the dataset: refine the system prompt or swap in a stronger LLM.
- After every change, re-run the same golden dataset; this is why evals are repeatable.
- Worked example: 80% → 90% (prompt tweak) → 95% (heavier LLM).
- Stop iterating when the system is judged worth deploying.

8. Step 9: Deploy and Monitor

Why Evaluation Continues After Deployment

When the system is satisfactory it is deployed online, but the work does not end. Without **monitoring**, a deployed system can still fail. A system that scored 95% on the test dataset may make mistakes when it sees new emails from real customers.

Turning Failures into Dataset Rows

When monitoring reveals a failure, for example an email that should have been labelled billing but was sent to the technical team, the procedure is:

1. Take that particular email (the failing instance).
2. Add it, with the correct label, to the **golden dataset**.
3. Restart the evaluation cycle on the enlarged dataset, improve the system, and redeploy.

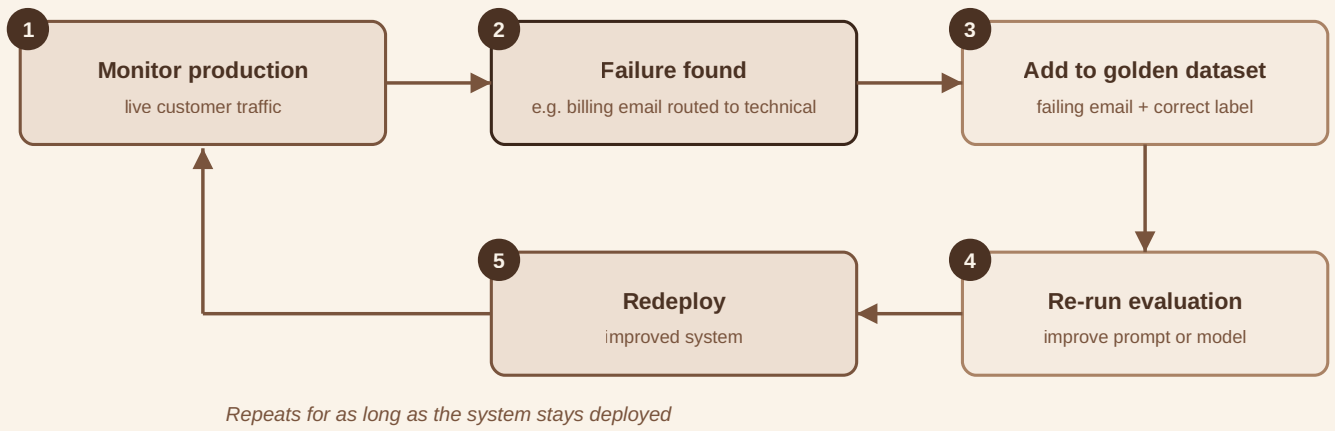


Figure 3.3 — Production failures are added to the golden dataset, so the dataset grows richer with each cycle.

Over time the golden dataset becomes richer, and the system is improved against it again and again. As the lecture puts it, this is how *reliable* LLM-based applications are built, and it runs for as long as the system is deployed.

Who Detects a Wrong Output?

A question raised in the lecture: during monitoring, who decides that an output was wrong? In the email example, a customer whose billing issue was sent to the technical team will tell that team they have the wrong issue. The technical team then **flags** the case as misrouted, and a flagging process in place feeds these cases back into the dataset. In short, a monitoring system with a defined flagging process must exist.

> **REAL WORLD INSIGHT**
>
> The loop in Figure 3.3 is the reason the golden dataset is never “finished”. Real failures are the most valuable test cases, because they are precisely what the previous version of the system got wrong.

↻ Quick Revision — Deploy and Monitor

- Deployment is not the end: a system accurate on the test set can still fail on new real data.
- Production failures are added to the golden dataset, and the cycle restarts.
- In the example, the receiving team flags misrouted emails so they can be fed back as new rows.
- Monitoring needs a process in place; it does not happen automatically.

9. Why One Application Can Have Several Evals

One more point flagged by the lecturer as important: a single LLM-based application may have **several LLM evals**. The email example used only one, but real applications are typically evaluated along more than one dimension.

Take a RAG application. On that single application you might run separate evals (Figure 3.4): one for the **retriever's** performance, one for the **embedding model's** performance, one for the **complete RAG workflow**, and one for the **latency** of the whole system.

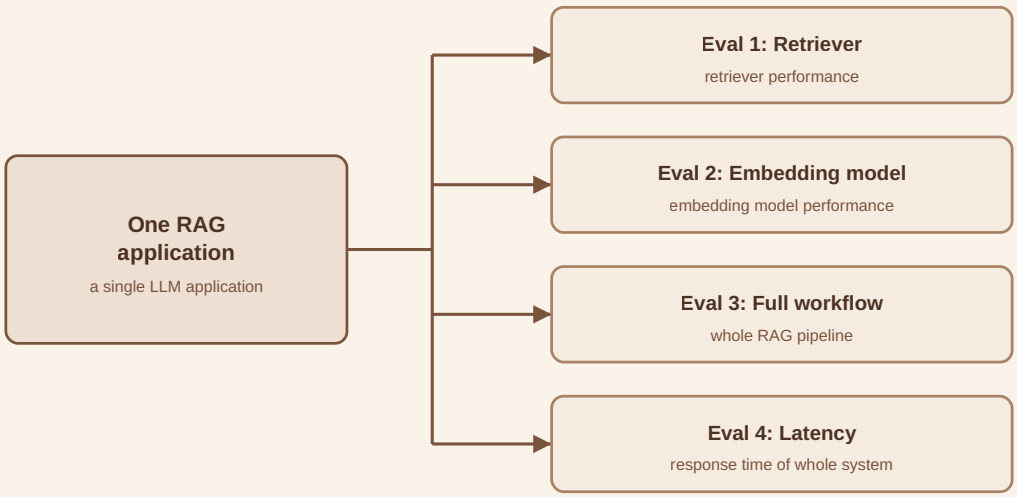


Figure 3.4 — One RAG application, several independent evals, each targeting a different part or property.

Each eval has its own target, metric, dataset, and method, but all follow the same nine-step workflow.

> **INTERVIEW TIP**
>
> If asked how you would evaluate a RAG app, do not answer with a single metric. Describe separate evals for the retriever, the embedding model, the end-to-end workflow, and latency, and say each follows the same workflow.

↻ Quick Revision — Multiple Evals

- One LLM application usually has more than one eval.
- RAG example: retriever eval, embedding-model eval, full-workflow eval, and latency eval.
- Each eval has its own target, metric, dataset, and method, but they share the same workflow.

K. Key Takeaways

- **Do not deploy untested.** Even a simple LLM app, such as an email router, must be evaluated before deployment.
- **Task, target, and success criteria.** Define what is being evaluated and what "working" means; for classification the metric is accuracy.
- **Golden dataset.** Manually labelled inputs and expected outputs, ideally from real data, typically 50–500 rows.
- **Evaluation method.** Automated code for label outputs; human or LLM evaluators when outputs are long free text.
- **Run, evaluate, analyze.** Send the dataset through the system, compute the metric, and study where errors occur.
- **Improve and iterate.** Change the system prompt or model, then re-run the same dataset; evals are repeatable (80% → 90% → 95% in the example).
- **Deploy and monitor.** Production failures are added to the golden dataset and the loop restarts for as long as the system is live.
- **Multiple evals.** One application can run several evals, such as retriever, embedding model, full workflow, and latency for RAG.

R. Revision Sheet

Evaluation Workflow

Nine steps: define task and target, define success criteria and metric, build golden dataset, choose evaluation method, run model, evaluate and analyze, improve, iterate, deploy and monitor.

Success Criteria and Metric

The criterion states what "working" means; the metric quantifies it. For the email classifier: correct routing, measured as accuracy (90 of 100 = 90%).

Golden Dataset

A manually labelled set of inputs and expected outputs (about 50–500 rows), best built from real past data and reused on every run.

Evaluation Method

Who scores the outputs: automated code, a human, or another LLM. Label outputs suit code; long text needs a human or an LLM.

Improve and Iterate

Change the system (prompt or model), then re-run the same dataset. Repeatability lets you compare scores across versions.

Production Monitoring

After deployment, watch for failures; add each failing example to the golden dataset and restart the cycle.

Multiple Evals per Application

A single application can have several evals, for example retriever, embedding model, whole RAG workflow, and latency.

I. Interview Questions

Q1. Why should a simple LLM application, such as an email classifier, not be deployed directly after being built?

Q2. List the nine steps of the LLM application evaluation workflow in order.

Q3. For the email-routing system, what are the task, the target, the success criterion, and the metric?

Q4. What is a golden dataset, how large is it typically in the lecture, and how should it be built?

Q5. When is an automated evaluation method sufficient, and when would you prefer a human or an LLM evaluator?

Q6. After the first run shows 80% accuracy, what two things could you change to improve the system?

Q7. What does it mean that LLM evals are repeatable, and why does the golden dataset need to stay fixed between runs?

Q8. Why is monitoring necessary after a system scores 95% on its test dataset?

Q9. How do production failures feed back into the evaluation process?

Q10. Name several evals you could run on a single RAG application and what each one targets.

F. Further Reading

- Anthropic Docs — *Define success criteria and build evaluations* (platform.claude.com/docs/en/test-and-evaluate/develop-tests).
- scikit-learn documentation — *accuracy_score* (sklearn.metrics).